

AI-based Village Pond Planning System

CSD Assignment 1 – Final Technical Report

SNEHA NAGMOTI, Indian Institute of Technology Bhilai, India

This report presents a web-based decision-support system for preliminary village pond planning. A user can select a study centre and radius on an interactive satellite map, or upload surveyed contour geometry in KML/KMZ form. The backend reconstructs a terrain surface, removes artificial depressions with Priority-Flood, resolves flats, computes D8 flow direction and accumulation, delineates the contributing watershed, and ranks spatially separated pond candidates. Satellite colour screening excludes detected surface water and identifies candidate open land. Historical rainfall is aggregated from complete calendar years; annual runoff is estimated by $V = C(P/1000)A$, with rainfall P in millimetres and area A in square metres. A geometry model converts usable runoff into pond depth, capacity, footprint and excavation dimensions while enforcing side-slope, freeboard and land-area limits. Results are returned as structured JSON and visualized as map overlays. In the verified live example, the system reported a 119.01 ha catchment, 1,324.2 mm annual rainfall, 472,758 m³/year runoff and three ranked sites; the selected design stores 378,206 m³. The delivered implementation includes a responsive React/Leaflet interface, a documented FastAPI service, public-data fallbacks, validation and rate limits, and automated backend and frontend tests.

Additional Key Words and Phrases: Village Pond Planning, Geospatial Analysis, Rainwater Harvesting, Catchment Estimation, Web GIS, REST APIs

1 Introduction

Seasonal rainfall can provide substantial local storage, but selecting a pond site is difficult because elevation, drainage, land cover and rainfall must be interpreted together. A site chosen only from a visual map can fall on a ridge, intercept little contributing area, overlap water, or require impractical excavation. The project therefore implements a geospatial web application that turns a selected land area or uploaded contour map into auditable pond-planning evidence. It produces a suggested site, watershed boundary, rainfall statistic, expected runoff, pond volume and dimensions, and renders these results directly on the map. The following sections define the requirements, architecture, hydrological method, implementation, CSD decisions and measured results.

1.1 Motivation

Manual screening is slow and inconsistent: contour interpretation is specialist work, rainfall records are distributed across services, and current land or surface-water conditions are not obvious from coordinates alone. Combining these inputs in one repeatable pipeline helps an administrator compare alternatives before commissioning a field survey. The tool is deliberately a screening system: it makes the data and assumptions visible instead of replacing engineering approval.

1.2 Scope of the Project

The system (i) supports map-based centre/radius selection and KML/KMZ contour upload; (ii) reconstructs terrain and drainage; (iii) identifies up to three ranked, separated pond options outside detected water; (iv) estimates catchment area, annual rainfall, runoff, storage capacity and excavation geometry; and (v) visualizes the watershed, candidates, elevation and drainage evidence. It does not certify land ownership, soil permeability, dam stability, flood safety or construction drawings. Final selection therefore requires cadastral, hydrographic, geotechnical and field verification.

Author's Contact Information: Sneha Nagmoti, Indian Institute of Technology Bhilai, Durg, India, snehan@iitbhilai.ac.in.

Manuscript submitted to CSD Assignment

2 Problem Statement and Requirements

The required system must accept a user-defined land area, analyze its terrain and rainfall, recommend a pond site, and display the site, catchment and expected water volume on a usable frontend. Table 1 maps each requirement to the implemented component.

Table 1. Functional requirements and implementation

Requirement	Implemented in
Satellite map and land selection	React/Leaflet map, search, coordinate and radius controls
Contour visualization/upload	Upload workspace and /api/analyze-contour
Candidate-land identification	RGB/HSV OpenCV screening with morphology and water mask
Catchment estimation	Priority-Flood, resolved-flat D8 and reverse watershed traversal
Historical rainfall	Open-Meteo archive with NASA POWER fallback
Runoff and pond sizing	terrain.py volumetric runoff and trapezoidal geometry
Combined result view	Toggleable Leaflet layers, ranked markers and result cards
Structured API output	Validated Pydantic response models and Swagger/ReDoc

2.1 Non-Functional Requirements

The application is responsive on current desktop and mobile browsers and keeps analysis work asynchronous so external calls do not freeze the interface. The API validates coordinate, radius, upload type, archive contents and size; applies per-route rate limits; restricts trusted hosts/CORS; and returns explicit source-quality metadata. The backend is stateless when optional history is disabled, so identical replicas can be placed on the four allotted systems. External failures use bounded timeouts, retries, cache and documented fallbacks rather than fabricated values. The production frontend is a static bundle, while health endpoints support availability checks. Target interaction latency is seconds for live screening and under one minute for a 15 MiB contour upload.

3 System Architecture and High-Level Design

The browser collects a location/radius or multipart contour file and calls the FastAPI service over HTTPS. Independent asynchronous adapters obtain elevation, satellite imagery and historical rainfall. The terrain and vision services normalize those inputs, the hydrology service computes drainage and candidates, and a sizing service produces runoff and pond geometry. Typed response models return both values and map geometry. Optional SQLite history is isolated from analysis and is disabled in the public demo; cache entries reduce repeated public-API calls. Figure 1 shows this flow.

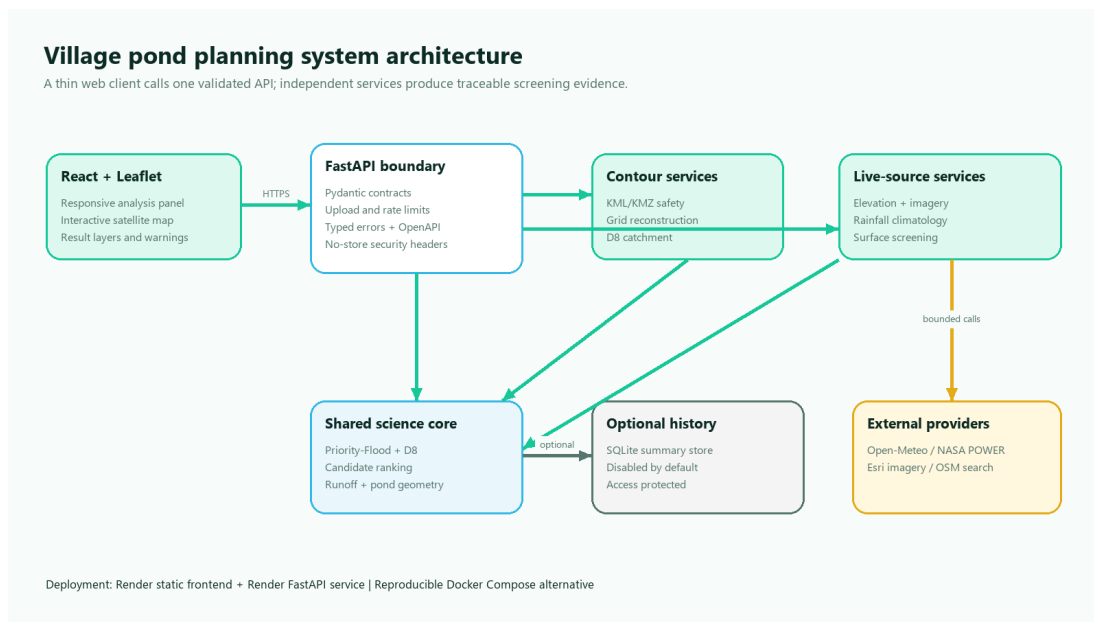


Fig. 1. High-level client, API, analysis-service and public-data architecture.

3.1 Technology Stack

The frontend uses React, Vite and Leaflet. The backend uses Python, FastAPI, Pydantic, NumPy, SciPy, OpenCV, Rasterio, Shapely, GeoPandas and HTTPX. Elevation comes from Open-Meteo/Copernicus GLO-90 compatible data with a bounded tile fallback; rainfall uses Open-Meteo Archive and NASA POWER; Esri World Imagery supplies the displayed and analyzed satellite mosaic. SQLite/SQLAlchemy provides optional local history. Pytest and Vitest test the two layers, and Docker/Render configurations provide reproducible deployment. This stack was chosen because its geospatial libraries implement the required raster/vector operations while FastAPI exposes an inspectable JSON contract.

4 Methodology

Both input modes enter the same hydrological core. Live mode samples an elevation grid over the chosen radius. Contour mode parses KML/KMZ safely, extracts line elevations, projects coordinates to metric space, rasterizes observations and performs fixed-observation harmonic interpolation. The latter iteratively replaces unobserved cells by neighboring means while never changing contour cells; convergence and observed-cell ratio are reported in the response.

4.1 Terrain and Elevation Analysis

The DEM is depression-filled with Priority-Flood so isolated numerical pits do not trap all flow. A small deterministic gradient resolves equal-height flats. Each valid cell then selects its steepest lower neighbour among the eight adjacent cells (D8). Processing cells in topological order gives flow accumulation in $O(n)$ after the $O(n \log n)$ flood, where n is the number of grid cells. Contours displayed by the frontend are regenerated from this surface. For uploaded data the original contour lines remain visible, making interpolation and source coverage auditable.

4.2 Catchment Area Delineation and Site Ranking

Candidate cells must lie inside the study/candidate-land mask, outside detected water and its buffer, away from the analysis boundary, and on a routable cell. A weighted score favors high log-flow accumulation (0.52–0.58 weight), low elevation, low local slope, interior clearance and distance from detected water. Non-maximum spatial separation prevents the top three options from collapsing onto the same point. For a chosen candidate, reverse traversal follows every D8 link that drains to it; the number of upstream cells multiplied by projected cell area gives the contributing area. Thus the reported catchment belongs to the selected pond point rather than to an unrelated study-area outlet. The drainage path is traced downstream for visual inspection.

4.3 Rainfall, Runoff and Pond Geometry

Daily precipitation is grouped into complete calendar years; incomplete years are excluded before computing mean annual rainfall. If the primary archive is unavailable, NASA POWER is queried with the same completeness checks. Annual runoff is

$$V = C \left(\frac{P}{1000} \right) A, \quad (1)$$

where A is catchment area in m^2 , P is annual rainfall in mm, and the course-demo coefficient is $C = 0.30$. The coefficient and its source are returned explicitly. A configurable capture fraction converts runoff to target storage. Pond length, width and water depth are solved for a rectangular basin with 2H:1V side slopes and 0.5 m freeboard; dimensions are reduced if the detected candidate land cannot contain the excavation footprint.

5 Implementation

5.1 Backend and API Design

The main routes are summarized in Table 2. The upload route accepts either `contour_file` or the assignment-compatible `contour_map` field; `/api/analyzeContour` and `/api/findCatchment` remain compatibility aliases. Validation errors use 4xx responses; third-party failures are bounded, recorded per source, and either fall back or produce an incomplete result instead of a false success. No login is needed for the public screening demo; history, when enabled, requires its configured access control.

Table 2. Public API endpoints

Method	Path	Purpose
POST	<code>/api/analyze</code>	Location/radius analysis
POST	<code>/api/analyze-contour</code>	KML/KMZ terrain analysis
GET	<code>/api/search-village</code>	Submitted place search
GET	<code>/health/live</code> , <code>/health/ready</code>	Service checks
GET	<code>/docs</code> , <code>/redoc</code>	API documentation

5.2 Frontend and Visualization

The interface offers Live Analysis and Contour Upload workspaces. A collapsible result panel leaves the map available for inspection. Users can select a location by search, coordinates or map interaction, set the radius, upload KML/KMZ, choose automatic/point/region contour selection, and switch among ranked options. Leaflet overlays show the study radius, source contours, reconstructed elevation surface, candidate land, water exclusion, catchment boundary, drainage path and numbered pond sites. Result cards present catchment, rainfall, runoff, capacity, depth, dimensions, score and source status in plain language. Figure 2 shows the verified live result.

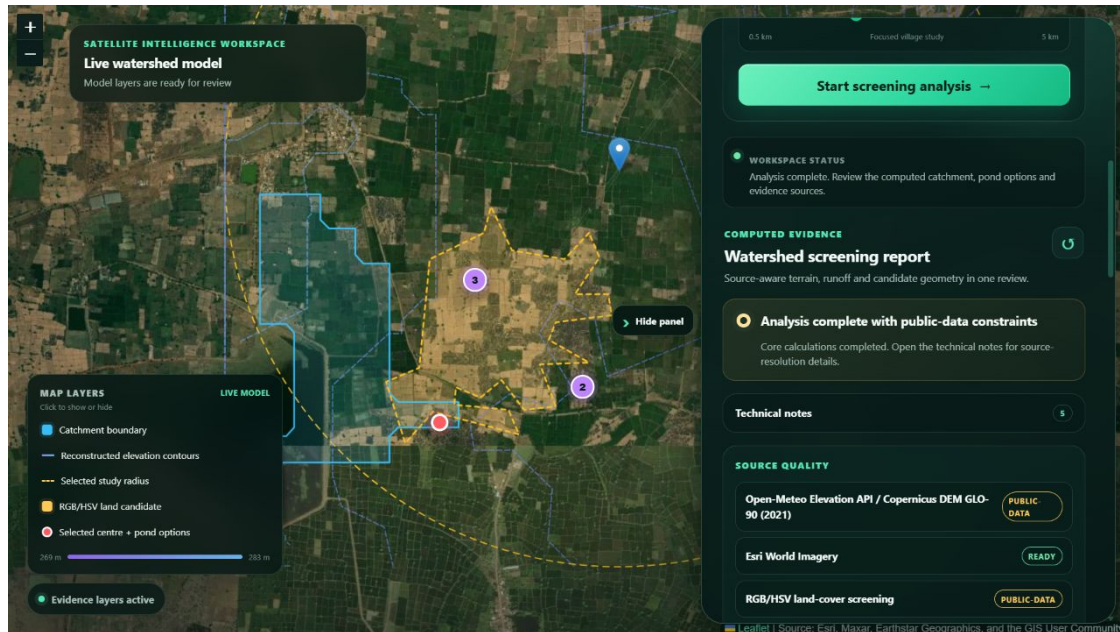


Fig. 2. Deployed live-analysis interface with catchment, candidate land and ranked sites.

5.3 Database and Storage

Analysis responses are self-contained and do not depend on persistence. A small SQLite store can retain selected summary records for a protected history view, but public deployment disables history to minimize stored user data. Public-source responses use bounded local caches with expiry; uploaded archives are processed in memory/temporary storage and are not used as permanent data.

6 CSD Themes and Topics Applied in the Project

The most important decision was a modular monolith rather than premature microservices: geospatial arrays remain in one process, avoiding serialization overhead, while source clients and algorithms still have clean interfaces. Asynchronous public-data calls shorten the critical path, while CPU-heavy terrain work remains deterministic and testable. Quality metadata is treated as part of the API contract; a successful HTTP call cannot silently turn missing rainfall or

Table 3. CSD concepts and concrete use

Theme	Application and rationale
REST contracts	Versionable typed requests/responses separate UI from analysis.
Concurrency	Elevation, imagery and rainfall are awaited concurrently to reduce wall time.
Caching	Repeated public-data queries reuse bounded TTL entries, reducing latency and quotas.
Modularity	Source adapters and hydrology/vision services can be replaced independently.
Resilience	Timeouts, retries, fallbacks and quality states expose partial-source failures.
Security	Upload/archive validation, limits, CORS, trusted-host and rate controls reduce attack surface.
Scalability	Stateless analysis and a static frontend allow independent replication on allotted hosts.
Testing/CI	Unit, integration and UI tests verify algorithms, contracts and Nginx security headers.

land evidence into a precise engineering answer. Because analysis is stateless, the same container can be replicated across the four laboratory systems and placed behind a reverse proxy if concurrency requires it.

7 Results and Evaluation

The deployed live workflow was manually verified at 21.244025°N, 81.288000°E. It produced three pond alternatives and the representative values in Table 4. The selected design stores 80% of annual modeled runoff and respects the configured depth, side-slope and footprint constraints. The supplied contours_1m.kml was also processed generically: 1,355 contours and 159,113 source points formed a 148×181 grid with 18 m cells; 85.206% of grid cells were directly observed and harmonic interpolation converged in 31 iterations. Its coloured DEM and ranked sites are shown in Figure 3. These are model-derived screening results, so correctness is evaluated by invariants, source traceability and map coherence rather than unavailable field ground truth.

Table 4. Representative live-analysis result

Measure	Result
Catchment area	119.01 ha
Mean annual rainfall	1,324.2 mm/year
Expected runoff ($C = 0.30$)	472,758 m ³ /year
Suggested pond location	21.250558, 81.296762
Suitability score	96.2 / 100
Water storage capacity	378,206 m ³
Water/excavation depth	4.0 / 4.5 m
Water dimensions	382.57 × 260.38 m
Excavation footprint	100,901 m ²

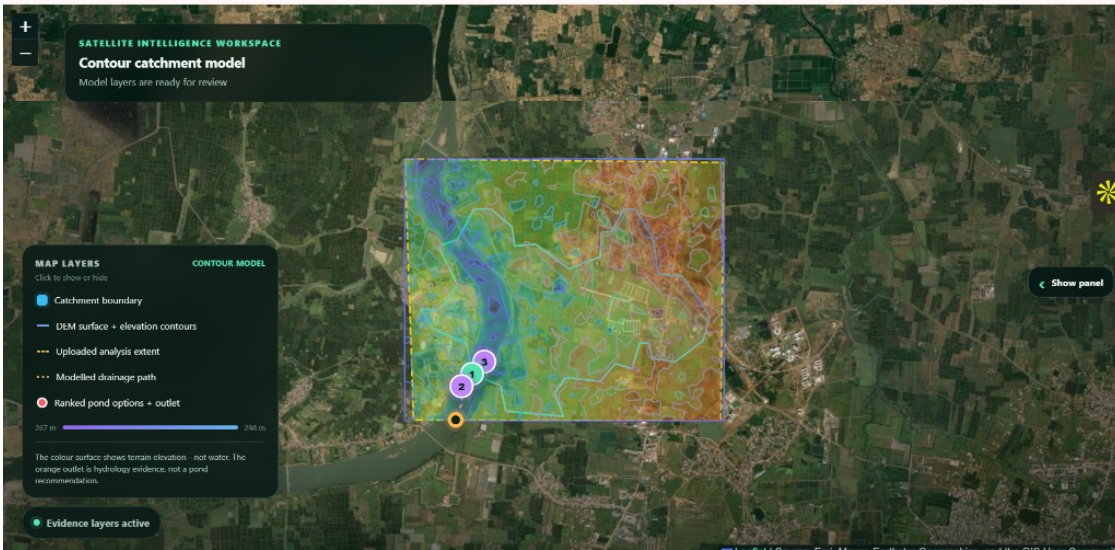


Fig. 3. Uploaded-contour DEM, catchment, drainage path and three ranked pond options.

7.1 Performance and Verification

The final backend suite passes 76 tests locally (7.24 s) and on the allotted system (14.14 s), including four independent reference-walk checks of flow accumulation and upstream catchment membership. The frontend passes 13 tests across six files; lint and production build succeed. The 118-module production bundle builds in 1.45 s, with main JavaScript and CSS assets of 447.69 kB and 41.94 kB before transfer compression. A full supplied-contour request through the college HTTP endpoint returned complete status in 31.58 s. A cold live-source request encountered unavailable imagery and correctly withheld a pond recommendation; a subsequent verified run completed with three candidates. Tests cover Priority-Flood/D8 behavior, point/region selection, water exclusion, contour parsing, upload limits, source failures, runoff/geometry constraints, map-result presentation and browser-facing routing. These are functional and numerical-consistency checks, not validation against a surveyed reference watershed.

On the allotted container's 512 MiB memory and one-CPU quota, two simultaneous supplied-KML requests completed successfully in 41.38 s and 59.05 s. Both produced the same 3,529,523.47 m² catchment and 1,355,474.66 m³/year runoff; an independent multiplication of area, rainfall and coefficient agreed with the reported volume. All 15 concurrent readiness probes succeeded, with a maximum response time of 0.271 s. This bounded concurrency test supports a small demonstration workload; it is not a sustained saturation benchmark or a high-availability claim.

7.2 Allotted-System Deployment and Recovery

The frontend and API run together on allotted system 2, accessed through SSH port 2238. External HTTP port 3238 forwards to its internal port 3000. The frontend uses relative /api requests, so the demonstration does not depend on the Render backend. User-owned Supervisor restarts an exited application process; this recovery was verified. A container reboot requires the documented start command because the container's initial process is SSH rather than a service manager. Successful satellite tiles are cached with a bounded lifetime, allowing missing-tile retries without accepting

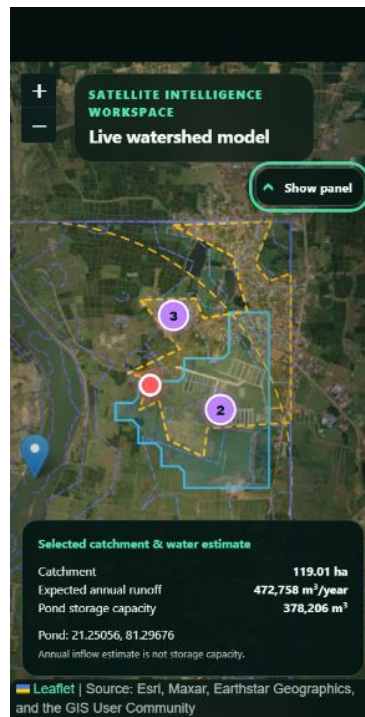


Fig. 4. Final allotted-system frontend: pond options and catchment with annual runoff and storage capacity directly on the map. The side panel is collapsed.

partial imagery. Intermittent college-network and upstream connection failures remain possible; source failures are shown explicitly.

The map displays the selected catchment boundary and pond options, with an on-map summary of catchment area, expected annual runoff, coordinates and storage capacity. Annual collected runoff and physical storage capacity are labelled separately. On narrow screens the summary remains visible with the controls panel collapsed. Deployment commands, resource checks and reproducible verification scripts are included in the repository.

8 Discussion and Limitations

The strongest result is traceability: every recommendation is accompanied by its watershed, ranked alternatives, source states and assumptions. Nevertheless, GLO-90-scale/public elevation cannot resolve small bunds and culverts; harmonic reconstruction between contours smooths unmeasured terrain; RGB/HSV screening may miss narrow, muddy, shaded or seasonal water; a uniform runoff coefficient ignores soil and land-use variation; and long-term annual rainfall does not replace design-storm routing. Land ownership is not inferred. The map therefore supports comparative screening, not construction approval. Future work should incorporate verified cadastral parcels, higher-resolution survey DEMs, official hydrography, soil-specific coefficients, IDF curves, spillway routing and measured validation sites.

9 AI Tool Usage Declaration

LLM tools were used for report drafting, formatting assistance, implementation assistance, test-plan review and assignment-requirement verification. Suggestions were checked against source code, automated test results, deployed API responses and map output. No LLM output was used as hydrological ground truth; terrain and runoff results are calculated from the input data and stated model assumptions.

10 Submission Links

- GitHub repository: <https://github.com/snehanagmoti/Village-Pond-planning>
- Allotted-system frontend: <http://10.1.75.53:3238/>
- Backend upload route (POST, file field contour_map): <http://10.1.75.53:3238/api/analyze-contour>
- Interactive API documentation: <http://10.1.75.53:3238/docs>

These private addresses require access to the college network. The public demonstration-video link is supplied separately with the submission.

A Source Code and Repository

The repository is available at <https://github.com/snehanagmoti/Village-Pond-planning>. The backend/ directory contains FastAPI routes, configuration, typed schemas, data-source adapters, hydrology/vision algorithms and Pytest tests. The frontend/ directory contains the React/Leaflet interface, Vitest tests and Nginx configuration. `render.yaml` defines the public deployment, while `docs/` and `phase3_report/` contain explanatory and submission material. The submitted revision is reproducible from the repository and contains no sample-specific hard-coded result coordinates.

References

- [1] Open-Meteo, "Historical Weather and Elevation APIs," <https://open-meteo.com/>.
- [2] NASA POWER Project, "Daily Meteorology API," <https://power.larc.nasa.gov/>.
- [3] Copernicus Programme, "Copernicus DEM GLO-90," European Space Agency.
- [4] Leaflet, "An open-source JavaScript library for mobile-friendly interactive maps," <https://leafletjs.com/>.
- [5] FastAPI, "FastAPI framework documentation," <https://fastapi.tiangolo.com/>.
- [6] R. Barnes, C. Lehman, and D. Mulla, "Priority-Flood: An optimal depression-filling and watershed-labeling algorithm for digital elevation models," *Computers & Geosciences*, vol. 62, pp. 117–127, 2014.